Salturi necondiționate și condiționate

- NECONDIȚIONATE : `jmp et`
  ↳ sare la eticheta et

- CONDIȚIONATĂ :

```
cmp op2, op1
j(condiție) etichetă
```

! `op1` este mereu registru!

j(condiție) poate fi:
- `jl` → jump if less than
- `jle` → jump if less than or equal
- `jg` → jump if greater than
- `jge` → jump if greater than or equal
- `je` → jump if equal
- `jne` → jump if not equal

Dacă `op2` este un registru:

ex.
```
cmp %eax, %ebx
jg etichetă
```

- Ce face? Calculează `%ebx - %eax` și este setat flag-ul
- Cum se citește?

Sari dacă %ebx > %eax (%ebx greater than %eax)
op1 op2

Scanat cu CamScanner

Dacă op2 este o constantă:

ex. cmp $10, %eax
jle etichetă

- Ce face? Calculează %eax - 10 și setează flag-ul
- Cum se citește?

Sari dacă %eax ≤ 10 (%eax less than or equal 10)
op1 op2 const.

- Ce este un flag? cmp op2, op1

Pt. instrucțiunea cmp, sub capotă, procesorul efectuează
- scădere internă fără a salva rezultatul într-un registru

rez_temporar = destinație - sursă
op1 op2

În urma acestei scăderi sunt actualizate flag-urile din registrul RFLAGS/EFLAGS:

- ZF (zero flag) → setat la 1 dacă op1 == op2 (diferența este 0)
- SF (sign flag) → setat la 1 dacă rezultatul scăderii este negativ (bitul cel mai semnificativ e 1)
- OF (overflow flag) → semnalează depășirea de semn
- CF (carry flag) → semnalează împrumutul (borrow) la operații fără semn.

Scanat cu CamScanner

- instrucțiunea loop → lucrează în %ecx prin dec lui

ex
```
    mov $10, %ecx
et_for:
    .......
    .......
    loop et_for
```

(echivalent cu un do...while)

→ verifică dacă %ecx == 0

DACĂ DA:
- merge la linia următoare

DACĂ NU:
- decrementează %ecx (%ecx = %ecx - 1)
- sare la et_for și reia

Scanat cu CamScanner

EXEMPLU program cu saltuni:

```
.data
  x:. long 8
  y:. long 10
  str1:. asciz "x<y\n"
  str2:. asciz "x>=y\n"
                1 2 3 4  5  + 1 = 6
.text
.global _start
_start:
  mov x, %eax
  mov y, %ebx
  cmp %eax, %ebx   -> compară ce se află în EAX, EBX
  jg et_afișare    -> ~~dacă (EBX < EBX)~~ dacă (EBX > EAX)
                      sare la et_afișare
                   altfel
                      următoarea linie
WRITE STR2
  mov $4, %eax
  mov $1, %ebx
  mov $str2, %ecx
  mov $6, %edx
  int $0x80
  jmp et_exit      -> se sare direct la exit ca să
                      nu afișăm șirul inelevant în
                      acest caz
et_afișare:
WRITE STR1
  mov $4, %eax
  mov $1, %ebx
  mov $str1, %ecx
  mov $5, %edx
  int $0x80
et_exit:
  mov $1, %eax
  mov $0, %ebx
  int $0x80        } EXIT
```

Scanat cu CamScanner

Simularea structurilor repetitive

În C++:	Assembly:
int sum = 0	mov $0, %eax
for (int i = 0; i < n; i++)	mov $0, %ecx
sum += i;	et_for:
	~~cmp %ecx, n~~
	cmp n, %ecx
	je et_exit
	add %ecx, %eax
	inc %ecx
	jmp et_for
	et_exit:
	-11-

Tablouri unidimensionale de date - pt întregi

4B 4B 4B 4B 4B 4B → 4B pt long poate fi și alte tip de date

V =	15	17	3	11	17	5
index:	0	1	2	3	4	5

- elementele sunt liniar așezate în memorie
- v reprezintă un nume pt adresa de la început
- pentru reținerea și folosirea vectorului se reține adresa în %ESI sau %edi (convențional)
- cunoaștem doar adresa de început și dimensiunea

v([b, c], d) → obține conținutul de la adresa de memorie

[b, c] - registri

d - constantă

Scanat cu CamScanner

b - base (de obicei, EDi sau ESi) - adresă

c - counter (indexul fiecăruia)

d - dimensiune (ne uităm la tipul de dată folosit și la cât spațiu ocupă)

ex. (%EDi, %ecx, 4)

↓ adresă ↓ counter ↘ dimensiune

calcul din spate: EDi + 4 · ECx

↙ adresa de început — fiind liniar și știind vectorul ne deplasăm spre fiecare elem.

Dacă vreau să accesez elementul cu indexul:

0 → (%EDi, 0, 4) → adresa primului element

1 → (%eDi, 1, 4) → adresa celui de al 2-lea

2 → (%edi, 2, 4) → -11- al 3-lea

etc.

Ca să reținem adresa vectorului [v] în EDi sau ESi se folosește: (v e doar numele variabilei ~~poate fi orice~~)

sau

```
lea v, %edi
```

sau

```
lea v, %esi
```

Declarare în memorie:

```
v: .long 15, 17, 3, 11, 17, 5
```

Scanat cu CamScanner

Să se calculeze suma din array-ul v

```
.data
  v: .long 10, 15, 30
  n: .long 3
.text
.global _start
_start:
  lea v, %edi
  mov $0, %ecx
  mov $0, %eax

et_for:
  cmp %ecx, n, %ecx
  je et_exit
  mov (%edi, %ecx, 4), %ebx
  addl %ebx, %eax
  addl $1, %ecx
  jmp et_for

et_exit:
  movl $1, %eax
  movl $0, %ebx
  int $0x80
```

Scanat cu CamScanner

```
.data
x: .long 0x4000002
z: .long 0x2000
t: .long 0b 100 0000 0000 0000 0000 = (4000)16
          baza2   40   0    0
.text
.global main
main:
    movl x, %eax    => EAX = 0x4000002
    mull z          => EAX = EAX · z = 0x4000002 · 0x2000
                       transformăm în puteri ale lui 2
et1:
    divl t
    decl %eax
et_exit:
    movl $1, %eax
    xorl %ebx, %ebx
    int $0x80
```

$$(4 \cdot 16^6 + 2) \cdot (2 \cdot 16^3) =$$
$$= (2^2 \cdot (2^4)^6 + 2) \cdot (2 \cdot (2^4)^3) =$$
$$= (2^{26} + 2) \cdot 2^{13} = 2^{39} + 2^{14}$$

trece de 32 biți deci

0x80 | 00004000
EDX EAX

EDX = 0x80
EAX = 0x4000

-> EAX) = EAX / t = ~~0x4000~~ / 0x4000 => EAX = 0x2000001
(EDX)
-> EAX = 0x2000000 => EDX = 0

Scanat cu CamScanner